

AGENTIC AI

A Complete Study Guide

Architecture · Frameworks · Memory · Tool Use · Multi-Agent Systems

Beginner Friendly

Framework Deep-Dive

Hands-On Examples

TABLE OF CONTENTS

1	What is Agentic AI?
2	Agents vs Chatbots
3	Agent Architecture
4	Function Calling & Tool Use
5	Agent Frameworks — LangChain & CrewAI
6	Multi-Agent Systems
7	Agent Memory
8	Building Your First Agent
9	Real-World Use Cases
10	Limitations & Safety Guardrails

1 What is Agentic AI?

Agentic AI refers to AI systems that can autonomously perceive their environment, plan multi-step strategies, take actions, use tools, and adapt based on feedback — all in pursuit of a high-level goal, with minimal human intervention at each step.

■ **Key Insight:** An agent is not just a model — it is a model embedded in a loop that can observe, reason, act, and learn.

Core Characteristics

- **Autonomy** — operates independently without step-by-step human guidance
- **Goal-directed** — pursues objectives rather than simply responding to prompts
- **Tool use** — can call APIs, run code, search the web, or query databases
- **Planning** — breaks large tasks into sub-tasks and sequences them logically
- **Memory** — retains context across multiple turns or even sessions
- **Adaptability** — revises plans when the environment changes or actions fail

The Agentic Loop

The core execution model of an agent follows a continuous Perceive → Think → Act → Observe cycle:

Step	Description	Example
Perceive	Gather inputs from environment	Read a user query + tool results
Think	Reason about the next best action	LLM decides to call a search API
Act	Execute the chosen action	HTTP GET to search endpoint
Observe	Record result and update state	Parse JSON; store in memory

Why Now?

- Large language models (GPT-4, Claude, Gemini) provide strong general reasoning
- Function-calling APIs let models invoke tools natively
- Open-source frameworks (LangChain, CrewAI) lower the barrier to entry
- Vector databases enable scalable long-term memory
- Cloud compute makes running long agentic loops affordable

2 Agents vs Chatbots

Although both chatbots and agents are powered by large language models, they differ fundamentally in capability, autonomy, and design philosophy.

Dimension	Chatbot	Agentic AI
Interaction model	Single turn or multi-turn conversation	Multi-step autonomous task execution
Tool access	Usually none or limited	Broad: web, code, APIs, files
Memory	Session context only	Short + long-term persistent memory
Planning	No explicit planning	Explicit task decomposition
Autonomy	Responds when prompted	Acts proactively on sub-goals
Error recovery	Requires user re-prompt	Self-corrects and retries
Output	Text response	Actions + artifacts + text

■ **Key Insight:** A chatbot answers 'How do I send an email?'. An agent actually sends the email, checks delivery, and reports back.

3 Agent Architecture

A production agent is built from three interconnected subsystems: **Planning**, **Reasoning**, and **Tool Use**. These run inside an orchestration loop driven by a core LLM.

3.1 Planning

Planning is the process by which an agent decomposes a high-level goal into an ordered sequence of smaller, executable steps.

ReAct (Reason + Act)

The most widely adopted pattern. The model alternates between a Thought (reasoning trace) and an Action (tool call), observing the result before continuing.

```
Thought: I need the current BTC price. I'll call the price API.  
Action: get_price(symbol="BTC")  
Observation: {"price": 62450, "currency": "USD"}  
Thought: Got the price. Now I can answer.  
Final Answer: Bitcoin is currently trading at $62,450.
```

Plan-and-Execute

A planner LLM first generates a full task list; an executor LLM then carries out each step. Useful for longer, more structured workflows.

Tree of Thoughts (ToT)

Explores multiple reasoning branches in parallel, evaluates each, and prunes weak paths — ideal for complex problem-solving tasks.

3.2 Reasoning

- **Chain-of-Thought (CoT)** — step-by-step reasoning improves accuracy on complex tasks
- **Self-reflection** — the agent critiques its own output and revises
- **Verification loops** — re-running logic checks or unit tests before finalising
- **Confidence scoring** — estimating uncertainty and escalating when unsure

3.3 Tool Use

Tools extend the agent beyond the model's parametric knowledge and turn it into a system that can affect the world.

- Web search & browsing
- Code execution (Python, JavaScript)
- Database read/write
- File system operations
- REST API calls (weather, calendar, CRM, etc.)
- Sending emails / messages

- Image and document generation

Architecture Diagram (Conceptual)

Component	Role	Example Implementation
LLM Core	Reasoning & decision-making	GPT-4o / Claude 3.5 / Gemini 1.5
Planner	Task decomposition	ReAct loop / Plan-and-Execute
Tool Registry	Available actions	LangChain tools, custom functions
Memory Store	Context persistence	Redis (short), Pinecone (long)
Orchestrator	Execution loop & retry logic	LangGraph, AutoGen, CrewAI
Output Handler	Format & deliver results	API response, file, UI widget

4 Function Calling & Tool Use

Function calling (also called tool use) is a native capability of modern LLMs that allows them to output structured JSON requests to invoke developer-defined functions instead of returning plain text.

How It Works

- **Step 1 — Define tools:** Describe each function with a JSON schema (name, description, parameters)
- **Step 2 — Send to model:** Pass the tool definitions alongside the user message
- **Step 3 — Model decides:** The LLM returns a `tool_call` JSON object when it wants to invoke a function
- **Step 4 — Execute locally:** Your code runs the function and returns the result
- **Step 5 — Continue:** The result is fed back so the model can reason further

OpenAI-style Example (Python)

```
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "City name"}
            },
            "required": ["city"]
        }
    }
}]

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Weather in Mumbai?"}],
    tools=tools
)

# Model returns: tool_calls=[{"name": "get_weather", "args": {"city": "Mumbai"}}]
```

Common Tool Categories

Category	Tools	Use Case
Information Retrieval	Web search, Wikipedia, RAG	Answer factual questions
Computation	Python REPL, calculator	Math, data analysis
Data Access	SQL, REST APIs, GraphQL	Query live data sources
Communication	Email, Slack, SMS APIs	Send notifications
File Operations	Read/write files, PDF parser	Document processing
Media Generation	DALL-E, Stable Diffusion	Create images/audio

■ **Note:** Always validate and sandbox tool outputs. An agent that can write to a database or send emails must have appropriate rate limits and approval gates.

5 Agent Frameworks — LangChain & CrewAI

5.1 LangChain

LangChain is the most widely adopted open-source framework for building LLM-powered applications and agents. It provides modular abstractions for chains, agents, memory, and tool integration.

Key Components

Component	Description
LLM / ChatModel	Wrapper around any model provider (OpenAI, Anthropic, etc.)
Prompt Template	Reusable, parameterised prompts
Chain	Sequential pipeline of LLM calls and transformations
Agent	LLM + tools + reasoning loop
Memory	ConversationBufferMemory, VectorStoreMemory, etc.
Tool	Any callable wrapped with a description for the model
LangGraph	Graph-based multi-agent orchestration layer

Minimal LangChain Agent

```
from langchain.agents import AgentExecutor, create_openai_tools_agent
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

@tool
def multiply(a: float, b: float) -> float:
    "Multiply two numbers."
    return a * b

llm = ChatOpenAI(model="gpt-4o")
agent = create_openai_tools_agent(llm, [multiply], prompt)
executor = AgentExecutor(agent=agent, tools=[multiply], verbose=True)
executor.invoke({"input": "What is 17 multiplied by 23?"})
```

5.2 CrewAI

CrewAI is a framework for orchestrating autonomous AI agents in role-based teams (crews). Each agent has a defined role, goal, and backstory, and agents collaborate via task delegation.

Core Concepts

- **Agent** — a role player with a goal, backstory, tools, and an LLM backbone
- **Task** — a specific piece of work assigned to an agent (with expected output)
- **Crew** — a collection of agents working together on a shared mission
- **Process** — Sequential (default) or Hierarchical (manager delegates to workers)

CrewAI Example

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(
    role="Research Analyst",
    goal="Gather and summarise AI news",
    backstory="Expert in tracking AI developments",
    tools=[web_search_tool],
    verbose=True
)

writer = Agent(
    role="Technical Writer",
    goal="Write a clear newsletter",
    backstory="Skilled at explaining complex topics"
)

research_task = Task(description="Find top 5 AI papers this week",
    agent=researcher)

write_task = Task(description="Write a 300-word newsletter",
    agent=writer)

crew = Crew(agents=[researcher, writer],
    tasks=[research_task, write_task],
    process=Process.sequential)

result = crew.kickoff()
```

Framework Comparison

Feature	LangChain	CrewAI
Primary focus	Chains, RAG, single agents	Multi-agent role-based teams
Abstraction level	Low-medium (flexible)	High (opinionated)

Feature	LangChain	CrewAI
Learning curve	Moderate	Gentle
Graph support	LangGraph extension	Built-in hierarchical process
Best for	Custom pipelines, RAG systems	Collaborative task automation
Community size	Very large	Growing rapidly

6 Multi-Agent Systems

Multi-agent systems (MAS) involve two or more AI agents that communicate, collaborate, or compete to accomplish tasks that would be difficult for a single agent to complete alone.

Why Use Multiple Agents?

- **Specialisation** — each agent is optimised for a narrow sub-task
- **Parallelism** — independent sub-tasks run concurrently
- **Scalability** — add more agents without re-architecting the whole system
- **Reliability** — agents can double-check each other (critic pattern)
- **Context window management** — distribute long tasks across agents

Communication Patterns

Pattern	Description	When to Use
Hub-and-Spoke	Orchestrator delegates to worker agents	Clear hierarchy, predictable tasks
Peer-to-Peer	Agents message each other directly	Loosely coupled workflows
Blackboard	Shared memory space all agents read/write	Complex coordination needs
Market-Based	Agents bid on tasks via an auction model	Dynamic load balancing

Design Patterns

Orchestrator–Worker

An orchestrator agent decomposes the task and assigns subtasks to specialised worker agents. Workers report back results; the orchestrator aggregates and decides next steps.

Debate / Critic Pattern

One agent proposes a solution; a second agent critiques it. Multiple rounds of debate improve accuracy on ambiguous or high-stakes decisions.

Reflection Pattern

An agent generates output, then a second 'reflector' agent reviews it for correctness, hallucinations, or style issues and suggests revisions.

■ **Key Insight:** AutoGen (Microsoft), LangGraph, and CrewAI all support multi-agent orchestration out of the box.

7 Agent Memory

Memory allows agents to maintain state, learn from experience, and personalise responses. It is divided into two broad categories: short-term (in-context) and long-term (external storage).

7.1 Short-Term Memory

Short-term memory lives in the model's context window — everything the model can 'see' in a single API call.

- **Conversation buffer** — full history kept in the prompt
- **Sliding window** — only the last N messages are retained
- **Summary memory** — older turns are compressed into a rolling summary
- **Token budget aware** — smart pruning to stay within context limits

7.2 Long-Term Memory

Long-term memory persists beyond a single session, typically stored externally and retrieved via semantic search or key lookup.

Type	Storage	Retrieval	Example Use Case
Episodic	Vector DB	Semantic similarity	Remember past user conversations
Semantic	Vector DB / KG	Embedding search	Domain knowledge base (RAG)
Procedural	Tool / code store	Name lookup	Reuse successful tool sequences
Declarative	SQL / key-value	Exact key lookup	User profile, preferences

Vector Database Landscape

- **Pinecone** — fully managed, production-grade
- **Weaviate** — open-source, multi-modal
- **ChromaDB** — local-first, great for prototyping
- **FAISS** — Facebook's in-memory similarity search library
- **pgvector** — vector extension for PostgreSQL

Retrieval-Augmented Generation (RAG)

RAG is the dominant pattern for connecting long-term memory to an agent. Documents are chunked, embedded, and stored in a vector database. At query time, the top-k most relevant chunks are retrieved and injected into the prompt as context.

```
# Simplified RAG flow
query_embedding = embed(user_query)
top_chunks = vector_db.search(query_embedding, top_k=5)
context = "\n".join(top_chunks)
prompt = f"Context:\n{context}\n\nQuestion: {user_query}"
```

```
answer = llm.invoke(prompt)
```

8 Building Your First Agent

We will build a simple research agent that searches the web, summarises findings, and writes a structured report. We use the OpenAI API + LangChain for clarity.

Step 1 — Environment Setup

```
pip install langchain langchain-openai duckduckgo-search python-dotenv

# .env

OPENAI_API_KEY=sk-...
```

Step 2 — Define Tools

```
from langchain_community.tools import DuckDuckGoSearchRun

from langchain_core.tools import tool


search = DuckDuckGoSearchRun()


@tool
def save_report(filename: str, content: str) -> str:
    "Save a report to a local file."
    with open(filename, "w") as f:
        f.write(content)
    return f"Saved to {filename}"
```

Step 3 — Create the Agent

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_openai_tools_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder


llm = ChatOpenAI(model="gpt-4o", temperature=0)


prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a research assistant. Search, analyse, and write reports."),
    ("human", "{input}"),
    MessagesPlaceholder("agent_scratchpad"),
])
```

```
tools = [search, save_report]

agent = create_openai_tools_agent(llm, tools, prompt)

runner = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=8)
```

Step 4 — Run the Agent

```
result = runner.invoke({
    "input": (
        "Research the latest advances in multimodal AI models in 2024. "
        "Summarise in 3 bullet points and save the report as report.txt."
    )
})

print(result["output"])
```

Step 5 — Add Memory

```
from langchain.memory import ConversationBufferWindowMemory

memory = ConversationBufferWindowMemory(k=5, return_messages=True,
memory_key="chat_history")

runner = AgentExecutor(agent=agent, tools=tools, memory=memory, verbose=True)

# Now the agent remembers the last 5 exchanges

runner.invoke({"input": "What did we discuss earlier?"})
```

■ **Key Insight:** Start with a small number of tools and short max_iterations. Add complexity incrementally once the basic loop is working reliably.

9 Real-World Use Cases

Agentic AI is already transforming multiple industries. Below are concrete, deployed examples across key domains.

Software Development

- **GitHub Copilot Workspace** — plan, code, test, and open PRs autonomously
- **Devin (Cognition)** — full software engineer agent: reads docs, debugs, deploys
- **Cursor / Aider** — multi-file code editing with context-aware agents

Customer Support

- Agents handle tier-1 support: look up orders, process refunds, escalate to humans
- Personalised responses by retrieving customer history from CRM via RAG
- Available 24/7 with consistent quality; handoff when confidence is low

Research & Knowledge Work

- **Elicit** — AI research assistant that searches, summarises, and extracts data from papers
- **Perplexity** — agentic search that cites sources and follows up automatically
- Internal knowledge management: agents index, tag, and surface company documents

Finance & Data Analysis

- Automated financial report generation from raw data files
- Anomaly detection agents that query databases and alert on outliers
- Portfolio optimisation agents that gather market data and rebalance based on rules

Healthcare

- Clinical note summarisation and ICD-10 code suggestion
- Drug interaction checking by querying medical databases
- Patient follow-up agents that schedule appointments and send reminders

Robotics & Physical Automation

- Warehouse robots that receive natural-language instructions
- Autonomous vehicles with LLM-based decision layers
- Smart home agents that coordinate IoT devices based on user preferences

Industry	Agent Role	Key Tools Used
E-commerce	Order tracking, returns, upsell	CRM API, payment gateway, email
Legal	Contract review, clause extraction	PDF parser, vector DB, code exec
Education	Personalised tutoring, quiz gen	Web search, code execution, LMS API
Marketing	Campaign creation, A/B copy gen	Web scraper, image gen, analytics API

Industry	Agent Role	Key Tools Used
IT Ops	Alert triage, auto-remediation	Shell exec, monitoring APIs, ticketing

10 Limitations & Safety Guardrails

Despite impressive capabilities, agentic AI systems have significant limitations and introduce new safety challenges that engineers must proactively address.

Technical Limitations

Limitation	Description	Mitigation
Hallucination	Model confidently states false facts	Retrieval grounding; output verification
Context window cap	Limited tokens constrain long tasks	Hierarchical summarisation; memory offload
Tool failures	APIs fail, return unexpected formats	Retry logic; graceful degradation
Infinite loops	Agent stuck in reasoning cycles	max_iterations guard; timeout watchdog
Latency & cost	Many LLM calls are slow and expensive	Caching; cheaper models for sub-tasks
Prompt injection	Malicious input hijacks agent behaviour	Input sanitisation; sandboxing

Safety Guardrails

Human-in-the-Loop (HITL)

Require human approval before irreversible actions: sending emails, modifying databases, making payments, or deleting files. Use confidence thresholds — if the agent is unsure, escalate.

Least Privilege Principle

Grant agents only the permissions they need for the current task. A research agent should have read-only web access — not write access to your production database.

Output Validation

- Run code in sandboxed environments (Docker, E2B) before execution
- Use a secondary LLM or rule engine to validate agent outputs
- Log all tool calls and intermediate reasoning steps for auditing

Rate Limiting & Budget Controls

- Cap the number of LLM calls per agent run (max_iterations)
- Set hard limits on API spend per session
- Implement circuit breakers for external service calls

Ethical Considerations

- **Accountability** — who is responsible when an agent makes a harmful decision?
- **Transparency** — users should know they are interacting with an AI agent
- **Bias amplification** — automated pipelines can scale biased outputs rapidly
- **Privacy** — agents with memory must handle PII in compliance with GDPR/CCPA

- **Misuse potential** — phishing, disinformation, or manipulation at scale

■ **Note:** Always define a clear 'kill switch' — a way to pause or terminate an agent run immediately without waiting for the current loop to finish.

Safety Checklist

- ■ Define clear task scope and success criteria before deployment
- ■ Use human approval gates for all irreversible actions
- ■ Sandbox all code execution
- ■ Log all LLM calls, tool invocations, and decisions
- ■ Test adversarial inputs (prompt injection, jailbreaks)
- ■ Set cost and iteration budgets
- ■ Implement graceful error handling and fallback paths
- ■ Conduct red-team exercises before production launch